智路OS项目 — 面试模拟问答（完整版）

> 每个问题提供【理解版】和【背诵版】两种答案。
>
> - **理解版**：帮你搞懂原理，面试前看这个来理解
> - **背诵版**：精简口语化，面试直接背这个
> - 背诵版中不出现具体函数名，用自然语言描述

第一条：设备抽象与插件化接入（相机设备）

简历原话

> 设备抽象与插件化接入：为相机等路侧设备定义统一的接入接口（初始化、启动停止、数据下发、状态查询），各厂商驱动实现同一套接口；新增设备型号时只需新增驱动文件并用一行注册宏注册，框架代码无需改动，接入方式从"改代码"变为"改配置"。

涉及的关键代码

① 统一接口（纯虚基类） — `base/device_connect/ipcamera/device_base.h`

```
class IpCameraDevice {
public:
    IpCameraDevice() = default;
    // 构造函数接收回调函数，设备采集到数据后通过它上报给框架
    explicit IpCameraDevice(const IpCameraCallBack &cb) : sender_(cb) {}
    virtual ~IpCameraDevice() = default;

    virtual bool Init(const std::string &config_file) = 0;  // 纯虚：初始化
    virtual void Start() = 0;                                  // 纯虚：启动采集
    virtual IpCameraDeviceState GetState() = 0;                // 纯虚：查询状态
    virtual void WriteToDevice(...) = 0;                       // 纯虚：下发控制命令

protected:
    IpCameraCallBack sender_;  // 回调函数，子类调它把数据传给框架
};
```

② 工厂类 + 注册宏 — `base/device_connect/ipcamera/device_factory.h`

```
class IpCameraDeviceFactory {
public:
    // 模板类：构造时自动把设备类注册到 map 里
    template <typename Inherit>
    class Register_t {
    public:
        Register_t(const std::string &key) {
            // 把 "设备名" → "构造函子" 插入 map
            IpCameraDeviceFactory::Instance().map_.emplace(
                key, [](const IpCameraCallBack &cb) { return new Inherit(cb); });
        }
    };

    // 根据设备名从 map 中取出实例
    std::unique_ptr<IpCameraDevice> GetUnique(const std::string &key,
                                              const IpCameraCallBack &cb);
    static IpCameraDeviceFactory &Instance();  // 单例

private:
    std::map<std::string, CONSTRUCT> map_;  // key=设备名, value=构造函子
};

// 注册宏：展开后创建一个 static 对象，构造函数自动执行注册
#define V2XOS_IPCAMERA_REG_FACTORY(T, key) \
    static IpCameraDeviceFactory::Register_t<T> 变量名(key);
```

③ 驱动文件末尾一行注册 — `middleware/.../standard_ipcamera/standard_ipcamera.cc` 第608行

```
// 就这一行！把 StandardIpCamera 注册到工厂，key 是 "standard_ipcamera"
V2XOS_IPCAMERA_REG_FACTORY(StandardIpCamera, "standard_ipcamera");
```

④ 组件层：加载 → 创建实例 → 初始化 → 启动 — `middleware/.../ipcamera_component.cc`

```
bool IpCameraComponent::Init() {
    // 第1步：扫描目录，dlopen 加载所有 .so，触发 static 对象自动注册
    DynamicLoader::GetInstance().LoadDevicePackage("device/lib/ipcamera/");

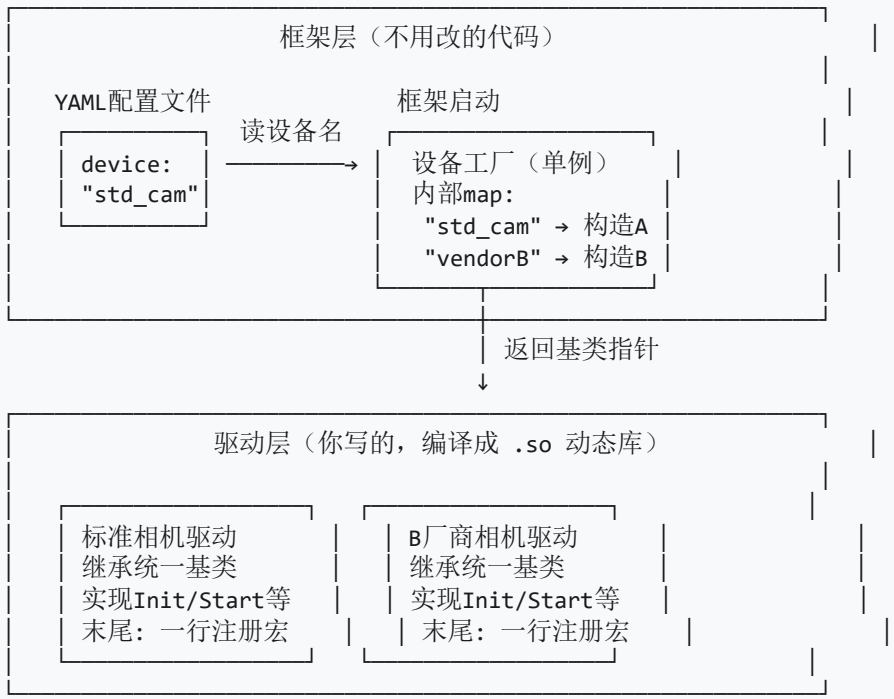
    // 第2步：读配置，拿到设备名（如 "standard_ipcamera"）
    LoadConfig(&conf_);

    // 第3步：从工厂根据设备名创建实例，同时传入回调函数
    device_ = IpCameraDeviceFactory::Instance().GetUnique(
        conf_.device(),                          // 配置里的设备名
        回调函数(MultiStreamCallBack));           // 数据往哪发

    // 第4步：调实例的 Init（实际执行 StandardIpCamera::Init）
    device_->Init(conf_.config_file());

    // 第5步：创建线程，调实例的 Start（实际执行 StandardIpCamera::Start）
    task_.reset(new std::thread([&]() { device_->Start(); }));
}
```

先搞懂你做了什么 —— 整体架构图



答题思路

① 痛点：多厂商设备，每来一个新型号就要改框架代码
↓
② 方案：统一接口 + 工厂模式 + 注册宏 + 动态加载
↓
③ 效果：新增设备 = 写一个.so + 改一行配置，框架零改动
↓
④ 细节：注册宏利用static变量构造函数，动态库加载时自动注册

Q1：请介绍一下你做的设备抽象与插件化接入

【理解版】

项目要对接多个厂商的相机，每个厂商的SDK和协议都不同。如果直接在框架里写if-else判断设备类型，每来一个新厂商就要改框架代码、重新编译。所以我设计了一套插件化架构：先定义一个纯虚基类（统一接口），包含初始化、启动、状态查询等方法，基类里还存了一个回调函数用来把数据上报给框架。然后做一个单例的工厂类，内部用map存"设备名→构造函数"的映射。各厂商驱动继承基类实现具体逻辑，文件末尾用一行注册宏把自己的类注册到工厂里。注册宏展开后会创建一个static的模板对象，C++保证static对象的构造函数在程序加载时自动执行，所以注册逻辑会自动完成。框架启动时扫描目录动态加载所有驱动的.so文件，然后从配置文件读设备名，去工厂里取对应实例。

【背诵版】

我为相机设备设计了一套插件化的接入框架。核心思路是四层：第一层是统一接口，定义了一个纯虚基类，包含初始化、启动、状态查询等方法，基类持有回调函数用于上报数据。第二层是工厂类，单例模式，内部维护一个map，key是设备名，value是对应的构造函数。第三层是注册宏，各厂商驱动文件末尾只需要写一行宏，就能自动注册到工厂里。底层利用了C++的static变量机制，static对象的构造函数在程序加载时自动执行，所以注

册逻辑不需要手动调用。第四层是动态加载，框架启动时扫描目录加载所有驱动的.so文件，然后从配置文件读设备名去工厂取实例。这样新增设备只需要写一个驱动文件加一行注册宏，改一下配置，框架代码完全不用动。

追问1.1：你说的"统一接入接口"具体包含哪些方法？

> 面试官从简历中"初始化、启动停止、数据下发、状态查询"自然追问

【背诵版】

四个核心方法。初始化负责读取配置、建立和设备SDK的连接。启动负责开始数据采集，内部会创建拉流线程。状态查询返回设备当前是运行中、已停止还是异常。数据上报是通过基类里存的一个回调函数，设备采集到数据后通过回调上报给框架。之所以统一成这几个方法，是因为所有相机设备不管厂商是谁，都需要经历初始化、启动、采集数据、查询状态这几个阶段，只是具体实现不同。统一接口后框架只需要面向基类编程，不关心具体是哪个厂商。

追问1.2：为什么用工厂模式而不是 if-else？

【理解版】

if-else的问题在于违反开闭原则——对修改不关闭。每来一个新设备就要在框架代码里加一个分支，改完还要重新编译整个框架。工厂模式把"选哪个设备"这个决策从编译时推迟到了运行时。框架代码只依赖抽象基类，不依赖任何具体实现。新厂商的驱动编译成独立的.so，运行时动态加载，框架零改动。

【背诵版】

核心原因是开闭原则。用if-else的话，每新增一个设备就要改框架代码再重新编译。而工厂模式把选择设备的逻辑从代码里移到了配置文件里。框架只依赖抽象接口，不依赖具体实现。新厂商的驱动是独立的.so文件，运行时动态加载，框架代码零改动。在多厂商多型号的路侧设备场景下，这个设计非常实用。

Q2：你简历上提到"一行注册宏"，这个注册宏是怎么工作的？

> 面试官从简历中"一行注册宏注册"自然追问

【理解版】

注册宏展开后会创建一个static的模板对象，这个对象的构造函数做的事就是把设备类的构造函子插入到工厂的map里。C++标准保证static对象的构造函数在程序加载时自动执行，所以当框架用dlopen加载驱动的.so文件时，这个static对象被构造，注册就自动完成了，不需要任何人手动调用。好处是各驱动完全解耦，新增驱动不需要在任何地方手动注册。

【背诵版】

注册宏展开后会创建一个static对象，这个对象的构造函数负责把设备类注册到工厂的map里。C++保证static对象的构造函数在程序加载时自动执行。所以当框架动态加载驱动的.so文件时，注册就自动完成了，不需要手动调用。新增驱动只要.so被加载，注册就自动完成。

追问2.1：这个统一接口是怎么实现的？C++里怎么做到"统一接口、不同实现"？

> 面试官从简历中"各厂商驱动实现同一套接口"自然追问

【理解版】

C++的虚函数是实现多态的机制。基类用virtual声明一个方法为虚函数，子类可以override重写它。当通过基类指针调用这个方法时，实际执行的是子类重写后的版本，这就是运行时多态。在这个项目里，设备基类把初始化、启动、状态查询等方法都声明为纯虚函数（=0），各厂商驱动继承基类后实现这些方法。框架持有基类指针，调用初始化时实际执行的是具体驱动的实现。这样就实现了"框架面向接口编程，驱动负责具体实现"的解耦。

【背诵版】

通过C++的虚函数和多态机制。设备基类把初始化、启动、状态查询等方法都定义为纯虚函数，各厂商驱动继承后实现具体逻辑。框架持有基类指针，调用时实际执行的是子类重写的版本，这就是运行时多态。这样就实现了框架和驱动的解耦——框架只面向基类编程，不关心具体是哪个厂商的实现。

追问2.2：纯虚函数和普通虚函数有什么区别？你为什么用纯虚函数？

> 面试官从追问2.1的回答自然追问

【背诵版】

普通虚函数是基类提供默认实现，子类可以重写也可以不重写。纯虚函数是基类不提供实现，标记为"=0"，子类必须重写，否则子类是抽象类不能创建对象。我用纯虚函数是因为每个厂商的设备初始化方式完全不同，基类没法给默认实现，必须强制子类去实现。如果用普通虚函数，万一某个厂商忘了重写某个方法，就会悄悄执行基类的默认实现，出了bug很难排查。纯虚函数在编译期就能强制你实现所有方法。

Q3：基类的析构函数有没有什么需要注意的？

> 面试官从Q2的多态讨论自然追问

【背诵版】

基类的析构函数必须声明为虚函数。因为框架用的是基类指针，如果析构函数不是虚函数，删除指针时只会调用基类的析构，子类析构不会被调用，子类里分配的资源就泄漏了。声明为虚函数后，删除时会先调子类析构再调基类析构，保证资源正确释放。这是C++中使用多态时的基本要求。

追问3.1：项目里怎么管理内存的？用了哪些智能指针？

> 面试官从你提到"基类指针"自然追问

【背诵版】

主要用了两种。unique_ptr用于设备实例的管理，工厂创建设备后返回unique_ptr，表示唯一拥有，不需要共享，零额外开销。shared_ptr用于数据传递，比如图像数据、消息等，因为这些数据可能被多个模块同时使用，

用引用计数保证最后一个使用者释放时才销毁。回调函数的参数也用shared_ptr，保证数据在回调期间不会被意外释放。

追问3.2：你的工厂类是单例的，怎么保证线程安全？

> 面试官从你提到"工厂类"自然追问

【理解版】

单例就是整个程序只有一个工厂对象，所有人操作同一个 map。实现方式是：构造函数私有化（外面不能 new），提供一个 static 的 Instance 方法，内部用局部静态变量返回唯一实例。C++11 保证局部静态变量的初始化是线程安全的。

实际代码（`base/device_connect/ipcamera/device_factory.cc`）：

```
// 工厂头文件里：构造函数是 private，外面不能 new
class IpCameraDeviceFactory {
private:
    IpCameraDeviceFactory() {}   // ← 构造函数私有化
    std::map<std::string, CONSTRUCT> map_;  // ← 所有注册信息都在这个 map 里

public:
    static IpCameraDeviceFactory& Instance();  // ← static 方法，不需要对象就能调用
};

// 工厂实现文件里：
IpCameraDeviceFactory& IpCameraDeviceFactory::Instance() {
    static IpCameraDeviceFactory instance_;  // ← 局部静态变量，只初始化一次
    return instance_;                         // ← 每次都返回同一个对象
}
```

这样不管谁调 `Instance()`，拿到的都是同一个工厂对象，操作的是同一个 map：

```
// 大华.so 注册时：
IpCameraDeviceFactory::Instance().map_["dh_camera"] = ...;
// 海康.so 注册时：
IpCameraDeviceFactory::Instance().map_["hk_camera"] = ...;
// 框架创建设备时：
auto device = IpCameraDeviceFactory::Instance().GetUnique("dh_camera", cb);
// 三次 Instance() 返回的是同一个对象，操作的是同一个 map
```

【背诵版】

构造函数私有化，外部不能new。提供一个静态方法获取唯一实例，内部用C++11的局部静态变量实现。C++11标准保证局部静态变量的初始化是线程安全的，多线程同时调用也不会创建多个实例。这是最简洁也最推荐的单例实现方式。

Q4：动态加载 .so 文件具体是怎么做的？

> 面试官从你提到"动态加载"自然追问

【背诵版】

Linux下用dlopen打开动态库。我们的加载器启动时扫描指定目录，对每个.so调用dlopen。加载时.so里的注册函数会自动执行，设备类就注册到工厂里了。如果dlopen失败，比如依赖库缺失，会返回空并打印错误日志。

追问4.1：如果两个驱动注册了相同的设备名会怎样？

【背诵版】

注册时会检查map里是否已经有同名key，如果已存在就跳过不覆盖，同时打印一条警告日志。不会出现后加载的驱动意外覆盖先注册的驱动的情况。

追问4.2：这个模块有什么不足？如果再做一次会怎么改？

【背诵版】

两个地方可以改进。第一，当前接口只定义了初始化、启动、状态查询等基础方法，但不同设备能力差异很大，比如有的相机支持云台控制有的不支持，统一接口不太好表达这种差异，可能需要引入能力查询机制。第二，注册宏虽然简洁，但调试不方便，如果.so没加载成功不容易定位。如果再做一次，我会加一个注册状态查询接口，加上更完善的加载失败日志和告警。

Q5：最终结果怎么样？怎么测试的？

【背诵版】

最终实现了两个厂商的相机驱动接入，接入第二个厂商时确实做到了框架代码零改动，只新增了驱动.so和修改了配置文件。测试分三步：第一步单元测试，单独加载驱动.so验证工厂能否正确注册和实例化；第二步集成测试，在完整框架中验证从设备初始化到数据发布的完整链路；第三步现场测试，在实际路侧设备上部署，验证长时间运行的稳定性。

追问5.1：测试中发现了什么问题？

【背诵版】

集成测试时发现了一个动态库加载顺序的bug。两个驱动.so，A和B，B驱动始终注册失败。排查发现B依赖了某个第三方库C，而C还没被加载，导致dlopen B失败。解决方案是加载.so前先检查依赖库是否已经在进程中，没有就先加载依赖。

追问5.2：还遇到过其他问题吗？

【背诵版】

还有一个线程安全问题。驱动在独立线程中采集数据，通过回调传给框架，但框架的消息发布不是线程安全的，多线程并发调用时会崩溃。排查过程是通过gdb看core dump的堆栈，定位到消息发布的方法。解决方案是在发送方法里加了一把互斥锁，保证同一时刻只有一个线程在发消息。

第二条：多路视频流接入

> ⚠️ **本条简历已精简**（换掉了旧版）。精简原则：删掉最难守的「状态标志与互斥锁」整句，把「FFmpeg 解码」改成「FFmpeg 拉流与拆包」，尾巴收短。 需要背的只有 3 句话 —— ①这模块干嘛的 ②核心怎么实现 ③我参与做了什么。其余都是储备。

简历原话

> 多路视频流接入：参与多路 RTSP 视频流的拉取与链路调试，每路视频流由独立线程处理，基于 FFmpeg 完成拉流与拆包并统一发布；支持断线自动重连与帧间隔统计。

涉及的关键代码

① 每路流的信息结构体 — `middleware/.../standard_ipcamera/standard_ipcamera.h`

```
class StandardIpCamera : public IpCameraDevice {
private:
    std::atomic<bool> stop_ = false;              // 全局停止标志（原子变量）
    std::vector<std::string> rtsp_urls_;          // 多路 RTSP 地址列表
    std::map<std::string, std::shared_ptr<StreamInfo>> streams_;  // 所有流
    std::mutex streams_mutex_;                    // 保护 map 的互斥锁

    // 每路流的独立信息
    struct StreamInfo {
        std::string rtsp_url;                     // 这路流的 RTSP 地址
        std::atomic<bool> stop;                   // 这路流的停止标志
        std::atomic<bool> connected;              // 这路流的连接状态
        std::unique_ptr<std::thread> pull_thread; // 这路流的独立线程
        AVFormatContext* format_ctx;              // FFmpeg 上下文
        uint64_t frame_counter;                   // 帧计数器
        // 帧间隔统计
        double last_send_time;                    // 上一帧发送时间
        double total_interval_sec;                // 累计间隔
        double min_interval_sec;                  // 最小间隔
        double max_interval_sec;                  // 最大间隔
    };
};
```

② Init：读配置、创建流信息 — `middleware/.../standard_ipcamera.cc` 第43行

```
bool StandardIpCamera::Init(const std::string& conf) {
    YAML::Node config = YAML::LoadFile(conf);
    // 读配置参数
    reconnect_interval_ = config["reconnect_interval"].as<int>(5);   // 重连间隔
    connection_timeout_ = config["connection_timeout"].as<int>(10);  // 连接超时

    // 读多路 RTSP 地址
    for (const auto& url : config["rtsp_urls"]) {
        rtsp_urls_.push_back(url.as<std::string>());
    }
    // 为每个 URL 创建 StreamInfo，存入 map
    for (size_t i = 0; i < rtsp_urls_.size(); ++i) {
        auto stream_info = std::make_shared<StreamInfo>();
        stream_info->rtsp_url = rtsp_urls_[i];
        streams_[output_topics_[i]] = stream_info;
    }
}
```

③ Start：为每路流创建独立线程 — `standard_ipcamera.cc` 第114行

```
void StandardIpCamera::Start() {
    std::lock_guard<std::mutex> lock(streams_mutex_);
    for (auto& pair : streams_) {
        // 每路流创建独立线程
        pair.second->pull_thread.reset(new std::thread([this, stream_id]() {
            this->TaskPullRtspStream(stream_id);  // 进入拉流循环
        }));
    }
}
```

④ 拉流核心循环 + 断线重连 — `standard_ipcamera.cc` 第225行

```
void StandardIpCamera::TaskPullRtspStream(const std::string& stream_id) {
    while (!stream_info->stop.load() && !stop_.load()) {     // 外层循环：控制重连
        if (!ConnectRtspStream(stream_id)) {                  // 连接 RTSP
            std::this_thread::sleep_for(seconds(reconnect_interval_));
            continue;                                         // 连接失败，sleep后重试
        }
        stream_info->connected.store(true);

        while (!stream_info->stop.load() && stream_info->connected.load()) {  // 内层：读帧
            int ret = av_read_frame(stream_info->format_ctx, &pkt);
            if (ret < 0) {
                stream_info->connected.store(false);          // 断线，退出内层
                break;
            }
            // 封装成 protobuf 消息
            auto compressed_image = std::make_shared<CompressedImage>();
            compressed_image->set_frame_id(...);
            compressed_image->set_data(h264_data);
            // 帧间隔统计
            double interval = current_time - stream_info->last_send_time;
            // 更新 min/max/total ...
            // 通过回调发给框架
            sender_(stream_info->output_topic, compressed_image);
            av_packet_unref(&pkt);    // 释放 packet，防止内存泄漏!
        }
        // 断线了，关闭连接，回到外层循环重连
        avformat_close_input(&stream_info->format_ctx);
    }
}
```

⑤ 组件层接收回调并发布 — `middleware/.../ipcamera_component.cc` 第74行

```
// 驱动通过回调把数据送到这里
void IpCameraComponent::MultiStreamCallBack(
    const std::string& stream_id, const IpCameraDataType& data) {
    Send(stream_id, data);  // 通过框架的 Writer 发布到消息通道
}
```

先搞懂你做了什么 —— 整体架构图

```
YAML配置 → 读取多路RTSP地址 → 为每路流创建StreamInfo + 独立线程

┌──────────────────────────────────────────────────────┐
│  StandardIpCamera（你写的驱动）                        │
│                                                      │
│  streams_ (map):                                     │
│  ┌──────────┬──────────┬──────────┬──────────┐       │
│  │ 流0 /cam1│ 流1 /cam2│ 流2 /cam3│ 流3 /cam4│       │
│  │ 线程0    │ 线程1    │ 线程2    │ 线程3    │       │
│  │ 独立状态 │ 独立状态 │ 独立状态 │ 独立状态 │       │
│  └──────────┴──────────┴──────────┴──────────┘       │
│                                                      │
│  每路流都执行:                                         │
│  连接RTSP → 读H.264包 → 封装protobuf                  │
│  → 回调发布 → 断线? → 重连                            │
└──────────────────────┬───────────────────────────────┘
                       │ 回调
                       ↓
              框架层接收 → 消息通道 → 下游感知模块
```

答题思路

```
① 背景：需要同时接入多路RTSP视频流（4路相机）
    ↓
② 架构：每路流一个独立线程 + 独立状态标志 + 共享map管理
    ↓
③ 流程：连接 → 读帧 → 封装protobuf → 回调发布 → 断线重连
    ↓
④ 亮点：断线自动重连、帧间隔统计、SEI时间戳解析
    ↓
⑤ 安全：mutex保护map，atomic控制启停，packet及时释放
```

Q1：请介绍一下多路视频流接入的整体方案

【理解版】

我负责的是标准IP相机驱动，处理多路RTSP视频流。启动时从YAML配置读取多路RTSP地址和输出topic，为每路流创建一个信息结构体（包含地址、FFmpeg上下文、帧计数器、连接状态、间隔统计等），然后为每路流创建独立线程。每个线程的流程是：用FFmpeg打开RTSP流，找到视频流索引，循环读H.264包，封装成protobuf消息（包含帧ID、时间戳、NALU数据等），通过回调发给框架发布。线程安全方面，用mutex保护流的map，每路流有独立的atomic标志位控制启停。断线时自动重连，每路流独立维护帧间隔统计。

【背诵版】

我参与的是标准IP相机驱动的开发，负责多路RTSP视频流的拉取和发布。整体方案是：启动时从配置文件读取多路RTSP地址和对应的输出topic，为每路流创建一个信息结构体，包含地址、连接状态、帧计数器、间隔统计等。然后为每路流创建独立线程。每个线程用FFmpeg打开RTSP流，循环读取H.264数据包，封装成框架标准的protobuf消息，通过回调发给框架发布。线程安全方面，用互斥锁保护流信息map，每路流有独立的原子标志位控制启停。支持断线自动重连，每路流独立统计帧间隔，方便现场定位链路问题。

追问1.1：你提到RTSP，能简单介绍一下RTSP协议吗？

> 面试官从简历中"RTSP视频流"自然追问

【背诵版】

RTSP是实时流传输协议，主要用于监控摄像头的视频传输。它基于TCP或UDP，支持双向控制比如暂停和跳转，延迟比较低，适合实时监控场景。工作流程是先通过RTSP协议建立连接、协商参数，然后摄像头推送或客户端拉取H.264码流数据。它是安防监控领域的行业标准协议，大部分IP相机都原生支持。

追问1.2：为什么每路流用独立线程而不是线程池？

【理解版】

RTSP拉流是阻塞IO，读数据的函数在没有数据时会一直等着。如果用线程池，一路流卡住就占住一个工作线程，卡住的流多了线程池被占满，其他正常的流就没人处理了。独立线程的话每路流互不干扰，一路卡住不影响其他流。而且独立线程配合原子标志位，可以精细控制每路流的启停，比如单独停止某一路。

【背诵版】

因为RTSP拉流是阻塞IO，读数据时没数据会一直阻塞。如果用线程池，一路流卡住就占住一个线程，卡住的流多了线程池被占满，其他流就没人处理了。独立线程每路流互不干扰，一路卡住不影响其他流。而且配合原子标志位可以精细控制每路流的启停。

Q2：这个模块的多线程安全是怎么保证的？哪些地方用了锁，哪些用了原子标志？

> ⚠️ 简历里已经删掉「状态标志和互斥锁」这句（太难守）。但多线程模块**大概率还是会被问到**，所以这是一道**储备题**——不需要写进简历，但被问到要能一句话答上来。

【背诵版】

互斥锁用在保护流信息的map上，因为map的插入、查找、遍历都是复合操作，必须加锁保证同一时刻只有一个线程在操作。原子标志用在每路流自己的停止标志和连接状态上，这些是单个布尔变量的读写，用原子操作就够了，不需要加锁，性能更好。简单说就是：复合数据结构用互斥锁，单个变量用原子操作。

追问2.1：互斥锁和原子操作有什么区别？

> 面试官从Q2的回答自然追问

【背诵版】

互斥锁保护一段代码或复合数据结构，同一时刻只有一个线程能执行，开销较大，涉及线程阻塞和唤醒。原子操作保护单个变量，保证读写不可分割，底层依赖硬件原子指令，不需要线程切换，开销更小。但原子操作只能保护单个变量的简单读写，不能保护一段复杂的逻辑。

追问2.2：断线重连的具体流程是什么？一直连不上怎么办？

【背诵版】

重连是两层循环。外层控制是否停止，内层负责读帧。内层因为断线退出后，先关闭当前连接，检查停止标志，不需要停止就sleep几秒，然后回到外层重新连接。一直连不上就一直重试，不会退出，每次失败都会打错误日志，运维人员可以通过日志发现问题。

Q3：你用 FFmpeg 具体做了什么？用了它的哪些功能？

> 面试官从简历中"基于 FFmpeg 完成拉流与拆包并统一发布"自然追问

【背诵版】

我用 FFmpeg 做的是**拉流和拆包**，不做像素级解码（不解码成 YUV 或 RGB）。主要用了三个功能：第一是打开 RTSP 流并获取流信息；第二是循环读取 H.264 数据包；第三是获取流的编解码信息比如视频流索引。读到的 H.264 裸数据直接封装成 protobuf 消息发布出去，解码交给下游感知模块按自己的需求去做。这样驱动层职责简单，不做多余计算。

> ❗ 注意：简历新版本写的是"**拉流与拆包**"，不再是"解码"。如果面试官追问"你是不是做了解码"，直接说：我用 FFmpeg 做的是拉流和拆包，像素级解码交给下游。

追问3.1：发布的是什么格式？为什么不做完整解码？

【背诵版】

发布的是H.264裸流数据，封装在protobuf消息里，包含NALU数据、帧ID、时间戳、是否关键帧等。不做完整解码是因为下游感知模块需要自己决定解码策略，有的要GPU硬件解码，有的要批量解码。驱动层的职责是保证拿到的原始数据完整、时间戳准确，解码交给下游做。这样职责清晰，驱动层不做多余计算。

追问3.2：帧间隔统计怎么做的？有什么实际用途？

【背诵版】

每路流独立维护上一帧发送时间，每发一帧就计算间隔，更新最小值、最大值和累计值。每100帧打印一次统计日志。实际用途是现场链路诊断。比如某路画面卡顿，运维看日志就能发现帧间隔最大值特别大，正常33毫秒一帧，最大值到了500毫秒，说明链路有抖动或丢帧。这个功能在实际部署中帮过几次忙。

Q4：SEI 时间戳解析是怎么做的？

> 面试官从简历中"并统一发布"追问：时间戳怎么来的？

【背诵版】

有些相机厂商会在H.264码流中嵌入曝光时间戳。做法是遍历H.264的NALU数据，找起始码，判断NALU类型，SEI的类型是6。找到后从固定偏移提取时间戳字符串转成整数。解析成功就用这个时间戳，失败就用系统当前时间。这个功能是可选的，通过配置开关控制。

追问4.1：这个模块有什么不足？如果再做一次会怎么改？

【背诵版】

三个地方可以改进。第一，帧间隔统计改为滑动窗口，只统计最近N帧的数据，更能反映实时状况，而不是从启动以来的全量统计。第二，增加阈值告警，帧间隔超过阈值时主动输出告警日志或触发回调，而不是等运维去翻日志。第三，把FFmpeg的超时和重连参数做成可配置的，不同网络环境不需要改代码就能调整。

追问4.2：最终结果怎么样？怎么测试的？

【背诵版】

最终实现了4路RTSP视频流的稳定拉取和发布，单路帧率约30fps。测试分三步：第一步单路流测试，验证拉流、封装、发布的完整链路，确认帧率和时间戳符合预期。第二步多路流压力测试，同时开4路以上，观察帧率是否稳定、有没有内存泄漏，用top看CPU占用，用帧间隔统计看有没有异常。第三步断线重连测试，手动断开某路流的RTSP源，观察是否能自动检测断线并周期重试，恢复后能否自动重连，同时验证其他路流不受影响。

Q5：遇到了什么问题或 bug？

【背诵版 - 问题一：内存泄漏】

开发初期4路流同时运行一段时间后内存持续增长。排查发现是FFmpeg读到的数据包在某些错误路径下没有释放，导致缓冲区一直累积。解决方案是在每次循环结束前统一释放数据包，确保无论正常还是异常都能释放。这个bug教会了我在用C语言的FFmpeg库时，一定要非常注意资源的成对释放。

追问5.1：还遇到过其他问题吗？

【背诵版 - 问题二：线程无法退出】

还有一个bug是调用停止时线程一直不退出。排查发现是连接RTSP流时没有设置超时参数，打开连接的操作默认超时很长，线程卡在连接操作上。解决方案是在打开流之前设置超时参数，这样最多阻塞指定时间就会返回，然后外层循环检查停止标志就能正常退出了。

追问5.2：还有没有小的问题？

【背诵版 - 问题三：帧间隔统计异常】

还有一个小问题，刚开始帧间隔统计的平均值偏差很大。后来发现是上一帧发送时间的初始值是0，第一帧发送时算出的间隔是从程序启动到第一帧的时间，可能好几秒，被算进了累计值。解决方案是加了判断，只有初始值大于0时才计算间隔，第一帧只记录时间不做统计。

通用追问准备

面试官问：你在这个项目中的角色是什么？

【背诵版】

我是实习生，主要参与了两个部分。第一是相机设备的插件化接入框架，我负责了接口设计、工厂模式实现和注册宏的设计，并实现了标准IP相机驱动。第二是多路RTSP视频流的拉取模块，我负责了多路流的线程管理、拉流逻辑、断线重连和帧间隔统计。框架整体架构是mentor设计的，我在他指导下负责具体模块的开发和调试。

面试官问：RTSP 和 HTTP-FLV 有什么区别？为什么选 RTSP？

【背诵版】

RTSP是实时流传输协议，支持双向控制，延迟低，适合实时监控。HTTP-FLV基于HTTP，只能单向拉流，穿透性好但延迟相对高。选RTSP是因为路侧监控需要低延迟，而且RTSP是安防监控的行业标准，大部分IP相机都原生支持。

面试官问：你在这个项目中学到了什么？

【背诵版】

三个方面的收获。第一是C++的工程实践能力，包括多线程编程、内存管理、动态库加载这些课本上学过但没真正用过的东西，在项目中有了实战经验。第二是系统设计思维，怎么把一个复杂问题拆解成接口、工厂、驱动三层，怎么做到新增设备不改框架代码。第三是调试能力，学会了用gdb看core dump、用日志定位问题、用帧间隔统计诊断链路异常。